Downloaded from orbit.dtu.dk on: Mar 30, 2026

DTU Library

Learning Joint Rebalancing and Dynamic Pricing Policies for Autonomous Mobility-on-Demand

Li, Xinling; Schmidt, Carolin; Gammelli, Daniele; Rodrigues, Filipe

Published in:
IEEE Transactions on Intelligent Transportation Systems

Link to article, DOI:
10.1109/TITS.2025.3582639

Publication date:
2025

Document Version
Peer reviewed version

Link back to DTU Orbit

Citation (APA):
Li, X., Schmidt, C., Gammelli, D., & Rodrigues, F. (2025). Learning Joint Rebalancing and Dynamic Pricing Policies for Autonomous Mobility-on-Demand. *IEEE Transactions on Intelligent Transportation Systems*, *26*(10), 16619 - 16634. https://doi.org/10.1109/TITS.2025.3582639

General rights
Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Learning Joint Rebalancing and Dynamic Pricing Policies for Autonomous Mobility-on-Demand

Xinling Li, Carolin Schmidt, Daniele Gammelli, and Filipe Rodrigues

***Abstract*— Rapid urbanization in the past decades has significantly escalated mobility demand and imposed higher service quality standards. As a promising solution to address this challenge, Autonomous Mobility-on-Demand (AMoD) systems offer tailored mobility services while facilitating centralized control. In this paper, we formulate the joint rebalancing and dynamic pricing problem in AMoD systems as a reinforcement learning problem over graph elements. By proposing a hierarchical policy, we exploit the benefits of both optimization and reinforcement learning methods. Through experiments conducted with real-world data from New York City and San Francisco, we demonstrate that, by leveraging the joint policy, the two control mechanisms can work in tandem to effectively address the limitations inherent to independent dynamic pricing or rebalancing policies under different scenarios. Crucially, we show that our approach is effective when learning from both (1) online interaction with the transportation system and (2) offline from static historical data, thus avoiding the potentially expensive interaction needed for training. The success of our proposed framework in improving system performance under different problem settings underscores its potential as a viable solution for controlling real-world transportation systems.**

***Index Terms*— AMoD, joint rebalancing and pricing, learning-based.**

I. Introduction

THE world has been undergoing rapid urbanization over the past decades [1]. The increasingly condensed population has led to a surge in the demand for mobility services, resulting in an explosion of private cars. However, private cars are widely considered an unsustainable solution for personalized mobility services as they significantly elevate energy consumption and carbon emissions. Additionally, overloaded private vehicles exacerbate congestion, leading to a waste of productive hours [2].

The adoption of Autonomous Mobility-on-Demand (AMoD) services has been increasingly proposed as a potential solution to address this challenge. However, effectively integrating AMoD into the current transportation network is a nontrivial task and requires extensive research and policy intervention. In AMoD, passengers request trips based on their demands, and the delivery services are dispatched to them accordingly. While the autonomous vehicles' fully compliant feature enables easier centralized control, it might result in a significant reduction of transit ridership in cities with mass transit [3], [4]. This issue could be mitigated through policies that integrate AMoD as a complement to, rather than a competitor with, traditional mass transit [3].

Another issue of AMoD is related to the rebalancing of vehicle fleets. One of the major challenges regarding AMoD systems control is the unbalanced distribution between vehicle supply and passenger demand, and rebalancing is essential in solving this problem to ensure AMoD service quality and profitability. However, rebalancing can increase vehicle traveling distance and contribute to congestion [4], [5]. Therefore, fleet control strategies must be rigorously designed to minimize these impacts and keep them within acceptable limits. Addressing the distribution mismatch requires a large amount of control and coordination.

The unbalanced distribution problem has been extensively studied in previous research. The solutions typically fall into two main categories: rebalancing and dynamic pricing. Rebalancing involves relocating vehicles to change supply distribution, while dynamic pricing aims to impact demand distribution by dynamically adjusting trip prices based on the current spatial features of transportation networks. Existing literature in these domains often revolves around formulating the problem through numerical optimization tools [6], [7], [8], [9], [10]. In this line of study, the systems are defined by a collection of constraints and the control goal is reflected by an objective function. The decisions are attained by optimizing the objective functions within the constraint set. While the construction of optimization models is relatively straightforward and intuitive, their low scalability has restricted their application in deriving real-time decisions for large-scale real-world systems.

Recently, reinforcement learning (RL) has emerged as a promising alternative that circumvents costly optimization routines. The fundamental concept behind RL approaches to the AMoD control problem is to learn a policy that determines optimal rebalancing flows or trip prices based on the current system state. Due to its high flexibility, there has been a growing amount of research centered around RL-based rebalancing [11], [12], [13], [14], [15] and RL-based dynamic pricing [16], [17], [18]. These works focus on learning policies that only consider one type of AMoD control

Received 10 September 2024; revised 1 February 2025 and 16 June 2025; accepted 17 June 2025. The Associate Editor for this article was Z. Ma. *(Corresponding author: Xinling Li.)*

Xinling Li is with the Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: xinli831@mit.edu).

Carolin Schmidt and Filipe Rodrigues are with the Department of Technology, Management and Economics, Technical University of Denmark, 2800 Kongens Lyngby, Denmark (e-mail: csasc@dtu.dk; rodr@dtu.dk).

Daniele Gammelli is with the Department of Aeronautics and Astronautics, Stanford University, Stanford, CA 94305 USA (e-mail: gammelli@stanford.edu).

Digital Object Identifier 10.1109/TITS.2025.3582639

1558-0016 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

strategy. As will be discussed later in this paper, solely implementing rebalancing or pricing can be too restrictive across different demand scenarios and misses out on potential synergies between the two strategies.

In this paper, we propose a centralized framework to *jointly* control rebalancing and dynamic pricing decisions within AMoD systems to balance the system demand and supply distributions. Specifically, we do not consider shared rides in one vehicle. Interested readers can refer to [19], [20], [21], and [22] for RL-based AMoD control with shared rides. As we empirically demonstrate, integrating both control mechanisms allows them to work in tandem, thus leading to more effective system control. Crucially, we show that our framework can learn effectively from both online interaction or solely from offline data.

The main contributions of this paper can be summarized as follows:

- We propose a novel framework that formulates *joint* rebalancing and dynamic pricing AMoD control from the perspective of RL. This framework is transferable, scalable, and highly sample-efficient due to innovative and step-wise validated design choices, such as the use of graph neural networks (GNNs) and the adoption of a bi-level control approach.
- We empirically show that the proposed joint policy outperforms independent dynamic pricing or rebalancing controllers across a heterogeneous set of scenarios.
- Based on our previous findings, we propose using offline RL to learn joint pricing and rebalancing policies. To the best of our knowledge, we are the first to demonstrate that complex policies for joint rebalancing and dynamic pricing can be learned offline. This has critical practical implications for real-world applications where static offline datasets are often readily available.

In the remainder of this paper, we first review the related literature on MoD rebalancing and dynamic pricing problems in Section II. In Section III, we provide an overview of the theories behind the RL and graph neural networks. In Section IV, we formally present the AMoD control problem. Section V shows the experiment results and analysis demonstrating the enhanced performance of the proposed online and offline learning methods in different scenarios. Finally, Section VI concludes the paper and suggests potential directions for future research.

II. Literature Review

The inconsistent distribution problem in MoD systems has attracted significant interest in the MoD control community over the past decades. In general, two main approaches exist to mitigate this problem: rebalancing and dynamic pricing.

A. Rebalancing

Rebalancing in MoD systems has been extensively studied using fluid models [23] and Model Predictive Control (MPC) [6], [7], [24], [25], [26], [27], where an optimization model is formulated based on network flow constraints and solved at each time step to yield the control decisions over a rolling horizon. In MPC, the optimization model takes consecutive intervals into consideration, while only the decisions for the first time interval are executed. These models usually assume that the true demand can be accurately predicted by an oracle, which is not a realistic setting. To deal with demand uncertainty, both data-driven approaches [7], [25] and robust optimization frameworks [28] have been discussed and applied. In [7], the robust optimization model with demand uncertainty interval proposed in [28] is compared with the data-driven sample average approximation approach. While the robust model is demonstrated to offer better solutions, it relies on explicit forecasts of future demand. Similarly, [25] builds a stochastic model based on a probabilistic forecast where a distribution assumption is necessary.

One crucial advantage of RL in transportation system control is its ability to deal with dynamic and stochastic environments. RL does not assume knowledge of system transition dynamics or its uncertainty. Recent research has started to explore the adoption of learning-based frameworks for rebalancing and demonstrated notable success. Reference [14] is among the first to apply RL to the MoD system rebalancing problem where the action is one of the eight cardinal directions. Based on the same discrete action set, [12] tested different RL algorithms. Reference [13] designed continuous action which is formulated as the desired idle vehicle distribution. The authors also proposed a cascaded learning framework to reduce the learning complexity. The rebalancing flows are determined by a minimum rebalancing distance problem based on the desired distribution. Reference [11] adopted the same idea in a three-step framework and used an actor-critic algorithm to learn a control policy. In the actor and critic neural networks, a GNN is applied to capture the spatial correlation of the network. Decomposing the training task into different steps helps the framework better scale up to large-scale systems [15]. Reference [29] extended this framework to offline RL to facilitate faster online fine-tuning.

In our work, we propose a framework that leverages a GNN architecture similar to the one proposed in [11] and adopts a bi-level approach as in [13]. However, crucially, while these two previous works only focus on rebalancing, we tackle the more complex joint rebalancing and pricing problem. By considering the joint problem, we advance the state of the art to handle a broader set of key operational decisions within AMoD systems. The joint decisions involve a larger action space with complex interactions between the two action axes (rebalancing and pricing) and require coordination for the two strategies to work in tandem.

B. Dynamic Pricing

Early research on dynamic pricing in MoD systems focuses on diverse approaches to model and predict the influence of the price on the system's profitability. Among the various methods, most of the researches rely on equilibrium-based [30], [31], [32], [33] and operations-research-based [8], [9], [34], [35] models. In the equilibrium-based approach, the system dynamic is modeled based on assumptions of system dynamics. These system dynamic assumptions lead to the definition of equilibrium conditions, which are presumed to exist. The pricing policy affects the equilibrium, which is subsequently

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

used to calculate the revenue. In operations-research-based works, the demand is typically considered elastic with respect to price, and this relationship is incorporated into the constraints set. Due to the complexity of the optimization models, specially designed optimization algorithms are required to efficiently derive optimal solutions, particularly for real-world-sized problems.

Different from the equilibrium or optimization-based models, RL is able to solve problems in stochastic environments with complex or unknown system dynamics. By moving the computational task partially to training which happens offline, RL helps improve online decision-making efficiency compared to the operations-research models, which is crucial in handling transportation system problems given their increasing sizes. Considering these advantages, RL is particularly well suited to dynamic pricing in AMoD systems.

There has been an increasing number of works seeking to leverage RL to solve dynamic pricing problems. Reference [36] adopted Q-learning to train the agent and demonstrated its superiority to static and a couple of other benchmark pricing schemes. Reference [16] applied a similar framework and demonstrated its contribution to resolving spatial service equity. More recent works have started to apply more stabilized RL algorithms for continuous action spaces. References [17] and [37] adopted an actor-critic method - proximal policy optimization (PPO) - to solve the optimal price problem. To mitigate the convergence problems due to the high dimensionality of OD-based decisions in large-scale systems, [38] proposed a multi-agent RL framework in which each region serves as an agent and only uses its own and neighbor's information as the state. Reference [39] sought to resolve the dimensionality problem from an algorithmic perspective. The authors introduced three mechanisms into an off-policy RL algorithm (TD3). While these works focus on OD-based price, we integrate the design of an origin-based price strategy in our framework and demonstrate that it achieves faster convergence without sacrificing system performance, which turned out crucial in making offline learning feasible.

Compared to rebalancing or pricing, the joint rebalancing and pricing problem in MoD systems has received substantially less attention during the past years. The joint optimization problem has been studied by mathematical programming [34], [40], [41] in car sharing systems, and more recently examined based on a fluid model [42], [43], following the spirit of [23]. For AMoD, [44] formulated the joint rebalancing and rebalancing problem into a profit-maximization problem for which the constraints are given by bounds of the system states. Due to the complexity and dependency of the state space, the authors proposed a relaxed version of the problem and solved it via a dual decomposition algorithm. Reference [45] approached the problem from a game-theoretical perspective and modeled it in the form of a dynamic mathematical program with equilibrium constraints (MPEC). The MPEC was solved by approximate dynamic programming (ADP) with piecewise linear value function approximation. Reference [46] also leveraged ADP to solve the joint control problem. The rebalancing, pricing, and charging decisions were embedded in the ADP framework and solved by value function updates.

In this paper, we explore learning-based approaches to the joint rebalancing and pricing problem. The advantage of the learning-based approaches over the optimization-based models is their ability to accelerate online decision-making by shifting part of the computational workload to the training process that is done offline. Findings from [11] suggest that while the MPC solution to the rebalancing optimization problem provides an upper performance bound, the optimization problem itself is NP-hard. This leads to scalability problems as the system scale increases. For RL-based controllers, even though the increased problem complexity requires longer training time, it does not significantly influence the efficiency of the online decision-making process. Consequently, the decision between using an optimization-based model and an RL-based controller should be seen as a trade-off between the offline training time and the online inference time. In this work, we focus on RL because it achieves better scalability when prioritizing online decision-making time.

Despite the success of RL in tackling MoD system rebalancing and dynamic pricing problems separately, a comprehensive study of their synergy within the framework of RL is notably absent. Related research is found in [37], where the authors applied RL for the joint optimization scheme of rebalancing and pricing in electric autonomous MoD systems. In [38], the authors extended the previous work to the multi-agent learning setting to further improve scalability. However, this line of work suffers from several major limitations. Firstly, the proposed framework by [37] relies on origin-destination (OD) based actions, resulting in a $\mathcal{O}(n^2)$ action space. This OD-based action space increases exploration complexity and may lead to convergence problems. Secondly, the authors utilized a Multilayer Perceptron (MLP) to approximate the value function and policy. This neural architecture cannot deal with transportation networks of varying sizes and connectivity, and it also overlooks the spatial correlations in the MoD system, leading to a data-inefficient learning process. Thirdly, the observed demand from the data is used as the maximum possible demand values, while, in reality, they correspond only to a censored version of the true demand [47] (i.e., limited by the available supply). This inconsistency could result in inaccurate analysis and conclusions drawn from the results.

In our work, we reduce the action space from OD-based to node-based by adopting a bi-level approach to improve scalability. The upper-level node-based action is transformed into the lower-level OD-based operations using an optimization model in the environment before they are performed. Additionally, unlike previous works on dynamic pricing, which focus on MLP-based architectures, we employ Graph Neural Networks (GNNs) as function approximators. GNNs are capable of exploiting the spatial correlation, facilitating faster convergence to enhanced performance, and enabling transferability of the learned policies across network topologies. Moreover, contrasting with previous works that mainly concentrate on analyzing the experimental results from different scenarios separately [16], [17], [18], [37], we conduct a comprehensive comparative analysis of the learned policies using data from various cities with different characteristics. This analysis helps demonstrate the reliability of the proposed framework under

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

different conditions and yields valuable insights regarding system control. We also show the transferability of the learned GNN-based policies to other unseen scenarios. Furthermore, we learn these policies using offline RL by exploiting the idea of conservative Q-learning [48]. To the best of our knowledge, this paper is not only the first to investigate the behavior of the joint rebalancing and dynamic pricing policies under different demand scenarios and their transferability across scenarios, but it is also the first to address this joint control problem through the lens of offline RL. Showing that these joint policies can be learned offline from historical data and can be transferred across scenarios/cities holds fundamental practical implications, opening up unique opportunities for the real-world application of the framework.

III. Preliminaries

A. Reinforcement Learning

Reinforcement learning is centered around acquiring skills that enable the mapping of states to actions in a dynamic system, with the ultimate goal of achieving a predefined objective. Intuitively, the fundamental concept of RL involves learning through experience by interacting with the environment. The mathematically idealized form of a dynamic system is the Markov Decision Process (MDP). It characterizes the dynamics of the systems through four elements $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{S}$ is a set of states $s \in \mathcal{S}$, $\mathcal{A}$ is a set of possible actions under the state, $\mathcal{P}$ describes the system dynamics transition function, and $\mathcal{R}$ defines a reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. For particular values of $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}$, the function $\mathcal{P}$ delineates the dynamics of the MDP by the following probability of these values occurring at time t:

$$\mathcal{P}(s'|s, a) = Pr\{S_t = s'|S_{t-1} = s, A_{t-1} = a\}, \tag{1}$$

The goal of RL is to learn a policy defining the distribution over possible actions given states, denoted as $\pi(a_t|s_t)$, with the objective to maximize the expected cumulative rewards following the learned policy. This expectation maximization problem can be formalized as follows:

$$\max_{\pi} \quad \mathbb{E}_{(s_t,a_t)\sim\pi}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right], \tag{2}$$

where T is the end time of a complete trajectory, and $\gamma \in (0, 1]$ is a scalar discount factor, indicating that the reward weights decrease over time.

The maximization problem (2) is typically addressed by alternating between policy evaluation and policy improvement until convergence is achieved. Policy evaluation aims to estimate the state or action value of the policy, while policy improvement entails making a new policy that improves on the original policy. The value function estimation is usually achieved by temporal-difference (TD) learning. For a given (s, a) and the next state s', TD learning updates the value function based on the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[R(s, a) + \gamma Q(s', a') - Q(s, a)\Big]. \tag{3}$$

B. Online and Offline Reinforcement Learning

In RL, the learning signals can originate from both the online interaction during the training process and datasets collected offline. To showcase that our proposed framework is agnostic to the learning signals, we train the agents in both online and offline settings. For online training, we leverage the soft actor-critic (SAC) [49] algorithm. SAC is a widely used deep off-policy actor-critic RL algorithm which is designed to achieve both sample efficiency and stability.

In online RL, the experience data is dynamically collected by making the agent interact with the environment. Ideally, this data should encompass actions of varied quality to prevent the agent from converging to a local minimum, a challenge commonly known as the exploration problem in RL. However, in many large-scale real-world systems, a highly sub-optimal action can lead to system failure and is highly unacceptable. In this case, the experience data is collected by interacting with an environment simulator. As the simulator provides only an approximation of a real-world system and cannot capture all the intricate details, a disparity inevitably exists between the simulator and the actual environment. The experience data collected based on the simulator may mislead the agent, causing it to learn inaccurate information and resulting in the formulation of a policy that suffers from sim-to-real distributional shift.

A solution to this problem is called data-driven RL, which utilizes only previously collected offline state and action data. As this method enables learning without online interaction, it is usually referred to as *offline RL*. Unfortunately, such data-driven offline methods pose the challenges of *distributional shift*: while the function approximator is trained under one distribution represented by the offline data, it might be evaluated on a different distribution. The approximator might overestimate the Q value of the out-of-distribution state-action pairs in the shifted distributions, and this is harmful to the policy.

The distributional shift problem has attracted significant research focus. For a comprehensive review of the different approaches, interested readers can refer to [50]. In this work, we exploit the idea of conservative Q-learning (CQL) [48] and Implicit Q-Learning (IQL) [51] for solving the distributional shift problem. CQL employs the learned policy to sample new actions based on the states in the dataset during training. It focuses on addressing the out-of-distribution sample overestimation problem. CQL modifies the Bellman backup equation (3) by including the following overestimation error to penalize the out-of-distribution actions:

$$\mathbb{E}_{s\sim\mathcal{D},a\sim\pi}[Q(s, a)] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s, a)], \tag{4}$$

where $\mathcal{D}$ is the static dataset consisting of state and action pairs.

IQL trains the agent only on the state-action pairs presented in the dataset, so it aims to mitigate the in-distribution sample underestimation problem. In online Q-learning, the learning objective is given by:

$$\min_{Q} \quad \mathbb{E}_{(s,a,s',a')\sim\mathcal{D}}\Big[(r(s, a) + \gamma Q(s', a') - Q(s, a))^2\Big], \tag{5}$$

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

IQL addresses the underestimation problem by leveraging the asymmetric squared loss instead of the least square loss for value function estimation to penalize positive and negative loss differently. The learning objective is modified to:

$$\min_{Q} \quad \mathbb{E}_{(s,a,s',a')\sim\mathcal{D}}\left[L_2^{\tau}(r(s,a)+\gamma Q(s',a')-Q(s,a))\right]. \tag{6}$$

where $L_2^{\tau}(u) = |\tau - \mathbb{1}(u < 0)|u^2$, which is an asymmetric loss function when $\tau! = 0.5$.

It is important to emphasize that the offline AMoD control learning adheres to the same MDP as the online one. The key distinction is that offline learning utilizes pre-existing offline data for training rather than collecting it through online interactions.

C. Graph Neural Networks

The remarkable representation and approximation capabilities of deep neural networks (DNNs) have elevated the efficacy of machine learning to unprecedented levels. DNNs significantly improve the learnable statistical properties of the data, such as stationarity and compositionality. Particularly, the MLP-based architecture has achieved substantial success in many machine learning tasks. However, when dealing with grid-like pixel data with MLPs, the order of the pixels makes a difference in the network output. This characteristic makes MLPs not generalize well to grid-like data, such as transportation networks, where the node ordering in the input data should not change any output. In other words, the neural network should exhibit *permutation invariance*.

To this end, a considerable body of research has focused on extending the idea of DNNs to non-Euclidean geometric data, such as graphs. The most widely applied neural networks for learning from graph data are graph neural networks (GNNs). Compared to MLPs, GNNs offer enhanced customization in terms of the number of nodes and the format of the adjacency matrix. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents the node sets and $\mathcal{E}$ the edge set, a GNN attempts to learn a permutation invariant function that takes as input the features of each node (typically summarized in a $N \times D$ matrix, where N is the number of nodes and D the number of features) and a representative description of the graph structure (typically in the form of adjacency matrix). The GNN then outputs an updated representation of all nodes in the graph.

A widely used GNN architecture for transportation networks is graph convolution network (GCN) [52]. For each GCN layer l, the propagation rule is as follows:

$$H^{l+1} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}H^l W^l\right). \tag{7}$$

where $\widetilde{A}$ is the adjacency matrix, $\widetilde{D}$ is a diagonal degree matrix and W^l is the layer-specific trainable weight matrix. $\sigma(\cdot)$ denotes the activation function and we use ReLU here. Intuitively, GCN describes a permutation invariant propagation rule that updates node-level features by aggregating information from neighborhood nodes through a shared local filter.

IV. The AMoD Control Problem

In this section, we delve into a detailed explanation on how we tailor these concepts to address the AMoD system control problem. We consider a transportation network that is discretized into N_v regions. Each region is represented by one central node, which is also assumed to be the location of the pick-up and drop-off station within this region. This network can then be represented by a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with M vehicles, where $|\mathcal{V}| = N_v$ and $(i, j) \in \mathcal{E}$ represents the set of edges characterizing the shortest path connecting the stations. We assume that each vehicle can carry at most one passenger at each time step. The time horizon is discretized into a set of intervals $\mathcal{T} = \{1, 2, \ldots, T\}$ of a given length ΔT. The vehicle travel time from station i to j is defined as the number of time intervals it takes by following the shortest path from i to j. This duration is represented as an integer $\tau_{ij} \in \mathbb{Z}_+$. The traveling cost on this edge at time t is denoted as c_{ij}^t, which is calculated based on the traveling time. The number of available vehicles at station i is denoted by m_i^j. At each time step t, customers arrive at their origin station i and wait in queue to be transported to their desired destination j at a rate d_{ij}^t. The vehicles serve the passenger on a first-come-first-serve basis, and the passengers have a maximum waiting time ω before leaving the system.

In the following part of the section, we start by formulating the AMoD pricing and rebalancing problem-based on the established system settings and model it as a MDP. Subsequently, we delineate the proposed GNN-integrated architecture to model the learning agent.

A. A Three-Step Policy

In this work, we adopt a bi-level approach to model the control policy. The idea of bi-level control is proposed in [53] to improve the RL agent scalability and the learning convergence performance in large-scale systems. For our problem, the bi-level approach translates into a three-step policy. The interaction between the policy and the simulation environment is illustrated by Fig. 1. Notice that this three-step policy allows us to formulate the action as node-based as opposed to OD-based, which reduces the action space from N_v^2 to N_v, thus significantly improving the scalability of the framework. More details regarding the compositions of the state and action will be elaborated on in the next section. Here, we only focus on explaining the policy and its interaction with the environment.

The three-step policy operates as follows: (1) the RL agent computes the price and desired idle vehicle distribution at the current time step, (2) the zone-based price multipliers are transformed into OD-based prices by multiplying them with their corresponding base prices obtained from the historical dataset. Further details regarding the conversion are explained in the next section, and (3) the node-based desired distribution is converted to rebalancing flows by solving a minimal rebalancing-cost problem, constrained by the desired vehicle distribution. Additional details about the optimization model can be found in Appendix B.

After the second and third steps, the actions determined by the policy influence the environment. Here, We elaborate on the components within the environment that are impacted.

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

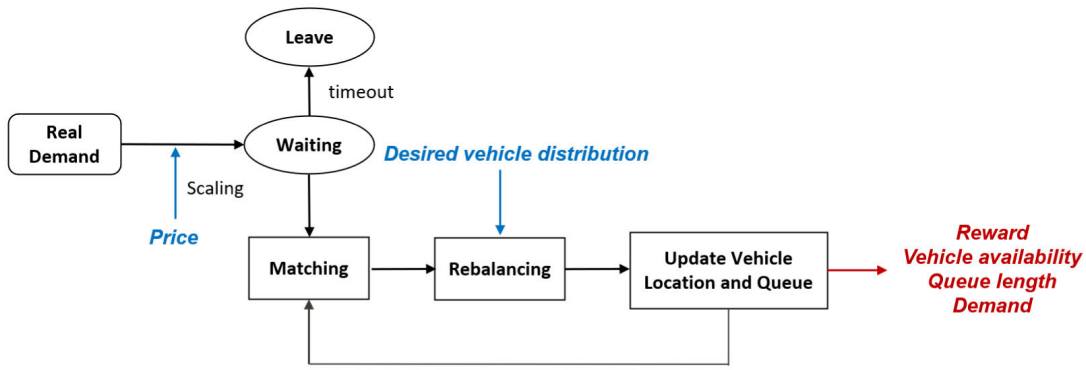

Fig. 1. Environment simulation workflow. The blue texts represent the input action from the learning agent, encompassing the price and the desired idle vehicle distribution. The red texts denote the returns from the environment following one simulation step, including the reward and the next state.

Specifically, after getting the OD-based price in step (2), the price is input to the environment, and the demand reacts to the price. Following that, a group of passengers is generated based on the price-adjusted demand. These passengers enter the queue at the corresponding departure station, waiting to be matched. The passengers are only matched with vehicles in the same region. There is a maximum waiting time for the passenger. The maximum waiting time is assumed to be homogenous across all the passengers in the system. The framework allows an easy extension of the heterogeneous waiting time for passengers without the need to modify any other parts. The vehicles are matched with the passengers in the queue of their current station in a first-come-first-serve manner.

Subsequently, after step (3), the OD-based rebalancing operations are conducted, and the vehicle location, as well as queue length at all stations, are updated accordingly. At this point, the system has transited to a new state, and one simulation step finishes. The new state and reward gained from the transition process are output to the agent.

B. The AMoD Control Problem as an MDP

In this section, we formulate the AMoD system control problem as a MDP. As discussed in Section III-A, MDPs are defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$. In what follows, we describe each of these elements in detail.

State ($\mathcal{S}$): The state consists of information that is necessary to determine the prices and rebalancing flows. This requires knowledge regarding the structure of the transportation network and additional station-level features of system supply and demand information. To this aim, at a given time t, we consider the state s_t as a composition of: (1) network adjacency matrix $\mathbf{A}$, (2) the currently available number of vehicles at each station $m_i^t \in [0, M], \forall i \in \mathcal{V}$, (3) the projected availability of idle vehicles along a planning horizon H $\{m_i^{t'}\}_{t'=t,\ldots,t+H}$, where this is estimated based on previously assigned passengers and rebalancing trips, (4) queue length at each station q_i^t, (5) current demand d_{ij}^t, and (6) current price p_{ij}^t. Crucially, we do not include estimated future demand in the state. However, this framework can be easily extended to include future demand estimates in states from an unbiased demand estimator such as in [47].

Action ($\mathcal{A}$): We consider the AMoD control problem involving determining the trip prices and rebalancing flows at the same time. The action consists of the price factor $p_i^t \in (0, 1]$ and the desired share of total available vehicle $w_i^t \in [0, 1]$ per area i, with the constraint that $\sum_{i=1}^{N_v} w_i^t = 1$. The joint action is denoted as $\mathcal{A}_t = [\mathbf{W}_t, \mathbf{P}_t]$, where $\mathbf{W}_t = [w_1^t, w_2^t, \ldots, w_{N_v}^t]$, and $\mathbf{P}_t = [p_1^t, p_2^t, \ldots p_{N_v}^t]$. $\mathbf{W}_t$ is related to the desired vehicle distributions which are used to calculate the rebalancing flows in the rebalancing optimization model mentioned in Section IV-A. We relate p_i^t to trip price p_{ij}^t on edge (i, j) by multiplying it with a reference price $\overline{p}_{ij}^t$:

$$p_{ij}^t = \beta \cdot p_i^t \overline{p}_{ij}^t, \tag{8}$$

where $\overline{p}_{ij}^t$ is estimated from historical trip data and β is the maximum price scalar. In our experiments, we set $\beta = 2$, which means the price for the edge (i, j) is upper-bounded by twice the reference price.

Reward ($\mathcal{R}$): We consider the reward from the perspective of an AMoD operator, for whom the objective is to maximize the profit. Thus, the reward function is defined as the difference between the revenue from serving passengers and the cost of operations. It can be expressed through the following formula:

$$r_{reb} = \sum_{i,j \in \mathcal{V}} x_{ij}^t (p_{ij}^t - c_{ij}^t) - \sum_{(i,j) \in \mathcal{E}} y_{ij}^t c_{ij}^t, \tag{9}$$

where x_{ij}^t and y_{ij}^t denote the passenger and rebalancing flows, respectively.

Dynamics ($\mathcal{P}$): The system dynamics describe the evolution of the system state. Upon selecting an action at time t, we need to determine how price and rebalancing decisions influence the state elements, including the demand patterns, vehicle availability, queue length, and current price. The prices are derived from the actions selected by the agent as described in equation (8). To get the demand at time step t, a reference demand $\overline{d}_{ij}^t$ and the corresponding reference price $\overline{p}_{ij}^t$ are first estimated from historical trip data. Then, under the current price p_{ij}^t set by the agent using (8), the demand d_{ij}^t is generated according to the following demand model:

$$d_{ij}^t = \overline{d}_{ij}^t \cdot \frac{p_{ij}^{max} - p_{ij}^t}{p_{ij}^{max} - \overline{p}_{ij}^t}, \tag{10}$$

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

where p_{ij}^{max} is the upper bound of the trip price from i to j with the constraint that $p_{ij}^{max} \geq \beta \cdot \overline{p}_{ij}^{t}$. This linear relationship between the demand and price is similar to [17]. This model implies a demand elasticity of -1.0 with respect to price, which is close to the taxi demand elasticities provided in [54].

As we focus on autonomous vehicle control, we can assume that they are fully compliant. Unlike human drivers who decide either to accept or reject the matched trip based on the price, autonomous vehicles always follow any decisions taken by the controller. The available number of autonomous vehicles in station i at t can then be calculated by:

$$m_i^t = m_i^{t-1} + \sum_{j \in \mathcal{V} \setminus \{i\}} arr_{ji}^t - \sum_{j \in \mathcal{V} \setminus \{i\}} veh_{ij}^{t-1} - \sum_{j \in \mathcal{V} \setminus \{i\}} reb_{ij}^{t-1}, \tag{11}$$

where m_i^{t-1} is the vehicle availability from the last time step, $\sum_{j \in \mathcal{V} \setminus \{i\}} arr_{ji}^t$ the projected number of incoming vehicles at time t (from both passenger and rebalancing flows), $\sum_{j \in \mathcal{V} \setminus \{i\}} veh_{ij}^{t-1}$ the vehicles matched with passengers at $t-1$, and $\sum_{j \in \mathcal{V} \setminus \{i\}} reb_{ij}^{t-1}$ the vehicles currently chosen to be rebalanced from i at $t-1$. Lastly, the queue length is defined accordingly by the following equation:

$$q_i^t = q_i^{t-1} + \sum_{j \in \mathcal{V} \setminus \{i\}} pax_{ij}^t - paxm_i^t. \tag{12}$$

where q_i^{t-1} is the queue length at the end of the last time step, $\sum_{j \in \mathcal{V} \setminus \{i\}} pax_{ij}^t$ is the incoming new passengers at time t, and $paxm_i^t$ is the matched passengers at time t. The adjacency matrix is independent of the selected actions and remains unchanged.

C. The GNN-Integrated Architecture

In this work, we propose a GNN-integrated architecture to model the actor and critic networks. To the best of our knowledge, we are the first to propose a GNN-based agent among the RL-based dynamic pricing literature. The architecture follows the actor-critic structure. The policy π and the action value function $Q_\theta(s_t, a_t)$ are parameterized by two networks, respectively. Both networks are characterized by a GCN layer followed by three fully-connected layers. The input features to the networks are the state introduced in Section IV-B. A figurative illustration of the architecture can be found in Appendix A.

In the policy network, the input state s_t is first encoded by a graph convolutional network (GCN) layer. The encoded state representation is obtained by adding up the encoded and original states, which is then fed into three fully-connected (FC) layers in sequence before the output layer. As introduced in Section IV-B, given a state s_t, the first component of the action is a vector of the desired idle vehicle distribution across stations, and the sum of this vector must equal to one. Thus, in order for π to define a valid probability density satisfying this requirement, we devise the first column in the output of the policy network as the concentration parameters $\alpha \in \mathbb{R}_+^{N_v}$ of a Dirichlet distribution. The distribution is then characterized by $W_t \sim Dirichlet(W_t|\alpha)$. The second component of the action is a vector representing the origin-based price scalar, with each element ranging from 0 to 1. To achieve this, we propose to model the price scalar as a Beta distribution, with the second and third columns in the policy outputs as two vectors of the parameters α' and β of the Beta distribution. The price scalar is derived by $P_t \sim Beta(P_t|\alpha', \beta)$. The positivity of all the aforementioned distributional parameters is guaranteed by the softplus activation function before the policy outputs.

The value function network adheres to a similar architecture as the policy network with the following modifications: (1) the action is concatenated with the encoded state representation before being input to the fully connected layers, (2) the output provides a single number instead of a vector.

V. Experiments

In this section, we evaluateCode available at: https://github.com/elxl/rl-pricing-amod our approach using the simulator proposed by [55]. This simulator incorporates calibrated scenarios from real-world datasets across various cities and regions. Specifically, we focus on scenarios from San Francisco, NYC Manhattan South (Man South), NYC Manhattan Middle (Man Middle), and NYC Brooklyn (Brooklyn). A more detailed description of the simulator and the statistics for the different available scenarios can be found in [55].

For our simulation, we set the maximum price parameter β to 2 and the maximum passenger waiting time as 6 minutes. $\beta = 2$ indicates that there will be no passengers if the price is twice as the base price in the dataset. Based on this setting, each scenario is trained with a time step of 3 minutes and an episode length 20 time steps. The scenario training setup aligns with previous research that utilizes the same simulator [11], [29], [55] to ensure consistency.

To demonstrate that the learned policy provides performance improvement, we evaluate it against four benchmarks:

- **No control (NC)**: The system runs without any rebalancing or dynamic pricing. After dropping off passengers, vehicles remain at the drop-off station.
- **Equal distribution (ED)**: At any time step, the system is controlled to achieve equally distributed available vehicles, regardless of the current system.
- **Proportional distribution (PD)**: At any time step, the system is controlled to redistribute vehicles proportional to the current passenger arrivals.
- **Rebalancing and pricing heuristic (Heu)**: The deterministic heuristic rebalancing and pricing policy is composed of an SDE-based heuristic for pricing [56] and a dynamic trip-vehicle assignment heuristic for rebalancing [57]. This heuristic calculates pricing and rebalancing decisions separately in sequence. Please refer to Appendix D for a more detailed description of the heuristics.

Additionally, the learned joint policy, which determines both the price and rebalancing operations, is compared to the following two policies:

- **Rebalancing**: The agent only decides the available vehicle distribution.
- **Dynamic pricing**: The agent only decides the price factor.

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

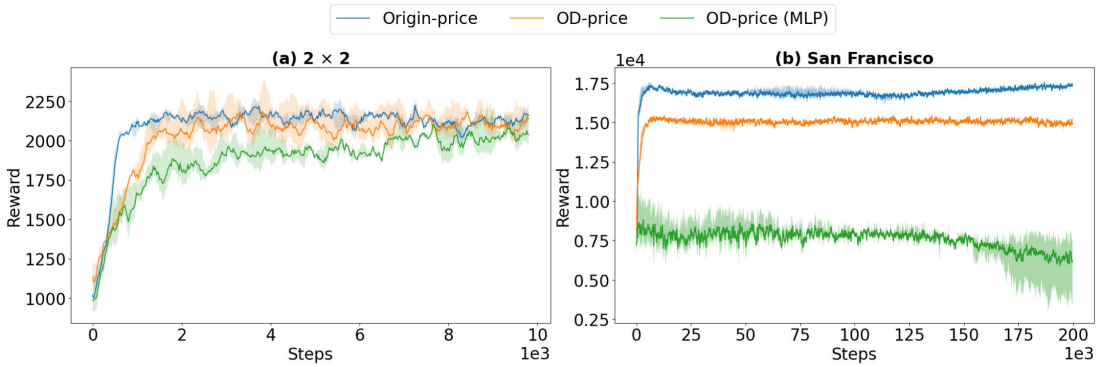

Fig. 2. Comparison between Origin-based and OD-based price. (a) Under 2×2 network, (b) under San Francisco network, which is 10×10. The x-axis represents the training steps, and the y-axis denotes the reward. The shades indicate the upper and lower bounds of three training runs for each setting. The origin-based price outperforms OD-price with GNN and MLP-based architectures when the system scale increases.

The rebalancing policy and dynamic pricing that the joint policy are compared with are both trained using the same environment as used to train the joint policy. This comparison of the three policies aims to demonstrate the efficacy of combining rebalancing and dynamic pricing in achieving a more effective AMoD system control.

As mentioned in Section II, [37] is the closest line of work to this paper. In the experiment, we compared our proposed framework with the one presented in [37]. Since the original implementation of the model proposed in [37] is not available, we do not conduct a direct comparison. Instead, we evaluate the different components in our framework by comparing them to the corresponding component in [37]. Concretely, in [37], the price is OD-based and the learning agent is modeled by MLPs. For comparison, in the remainder of this section, we start by validating the effectiveness of origin-based pricing and GNN-integrated agent architecture. The origin-based and GNN-integrated design offers better scalability and convergence performance compared to [37]. Following that, based on our proposed framework, we conduct experiments on the online and offline AMoD control problem using data from different real-world scenarios.

A. Comparing Origin-Based and OD-Based Pricing

The design of an origin-based price is pivotal in reducing the dimensions of the action space, thereby enhancing the scalability of the framework. Despite the prevalent use of origin-based pricing in RL-based dynamic pricing, a comprehensive justification for this choice and its impact on learning outcomes in contrast to OD-based pricing strategies has been lacking. In this section, we validate our proposition of the origin-based price through a direct comparison with OD-based pricing. Specifically, the origin-based pricing derives the pricing scaler for each node from the node's features, while the OD-based pricing predicts the scalar for each edge considering features from both its origin and destination.

We start the comparative analysis by comparing the origin and OD-based pricing mechanisms in a hypothetical scenario featuring a 2×2 network with time-dependent demand. In this comparative analysis, we also benchmark the two policies against a third one following the pricing method and policy network structure proposed by [37]. In this approach, the price is OD-based, and the policy is approximated using MLP. The choice of a small-scale case aims to discern the performance differences without the complexities introduced by high dimensionality.

Fig. 2(a) shows that the convergence states of these three policies are similar. It is essential to note that, in our problem setting, we leverage real data to estimate the base price for each OD pair and rescale it by the scalars provided by the agent. This approach implicitly incorporates trip distance and travel time, which have an impact on the trip price and depend on the destination, into the estimated price. As a result, with origin-based pricing, although the origin-based price scaler is uniform across all trips originating from an origin, the trip prices vary due to the difference in the estimated base prices.

Fig. 2(a) demonstrates that the origin and its neighbors provide sufficient information for determining the trip price originated from it. Subsequently, we augment the problem size to a real-world scenario, comparing the two approaches utilizing data from San Francisco with a 10×10 network.

As depicted in Fig. 2(b), our proposed approach of origin-based pricing with GNN achieves the highest converging state. Compared to OD-based pricing, origin-based pricing significantly reduces the action space and mitigates the problem of the "curse of dimensionality" under increasing problem scale. In contrast, for OD-based pricing, the large action space requires more extensive exploration during the training process, leading to a slower convergence speed. For the MLP-based OD-based pricing, the insufficient capacity of MLP to effectively learn information from graph data exacerbates the learning speed and convergence issues. This result highlights the limitations of the approach proposed in [37], namely its low scalability under increasing problem size. The results presented in Fig. 2 show that, in our RL-based problem context, the adoption of the origin-based pricing strategy does not compromise agent performance in the application of the proposed framework and is of higher scalability in comparison to OD-based price. Therefore, throughout the remainder of this paper, we maintain the use of origin-based pricing across all experiments discussed.

B. Comparing GNN and MLP

In SAC, the actor and critic are parameterized by neural networks. While the theoretical proof of convergence for the algorithm [49] remains unaffected by the choice of neural

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

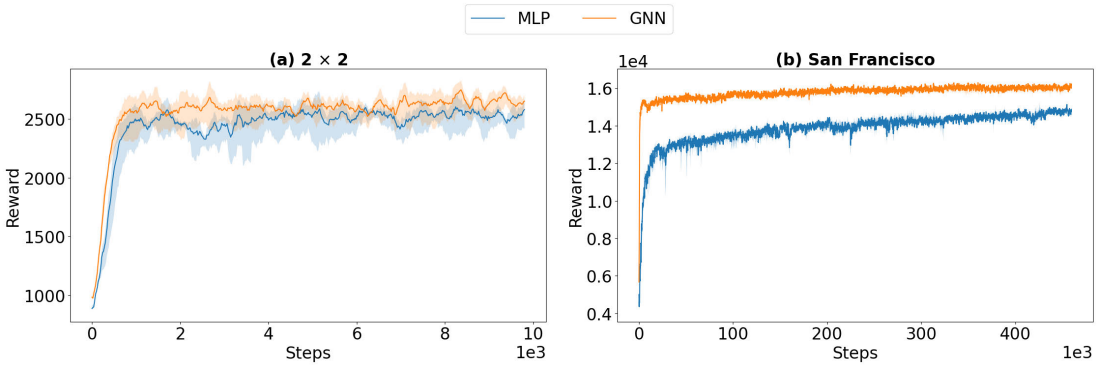

Fig. 3. Comparison between GNN-integrated and MLP-based actor-critic. The two architectures converge at a similar rate in small case, but the MLP-based agent converges significantly slower when system scale increases.

network architecture, its practical impact on learning outcomes is substantial. Different neural network structures are tailored to focus on different aspects of representation learning and information propagation. In our experiments, we opted for GNN to model both the actor and critic as delineated in Section IV-C. In this work, we aim to prove the benefit of permutation invariance in graph learning. Comparing different permutation-invariant structures is out of the scope of this research. Interested readers can refer to [58] and [59] for a more comprehensive comparison of the possible structures.

An alternative widely employed neural network architecture in this context is MLP. Due to its intuitive architecture, MLPs have been extensively used in RL-based transportation system networks and demonstrated efficacy [17], [18], [37]. Despite the success of MLPs, we demonstrated the superiority of GNN through our experiments. GNN is specially designed for graph data, enabling the *permutation invariant* property that aligns with the characteristics of transportation networks. The MLP-based neural network differs from the architecture presented in Fig. 6 by replacing the GCN layer with another fully-connected layer. Aside from this modification, the input and output architecture remains identical.

The comparative result is displayed in Fig. 3. Similar to Section V-A, we conduct the analysis in both the hypothetical 2×2 and San Francisco cases. Fig. 3 illustrates that the MLP-based actor-critic exhibits a significantly slower learning speed in comparison to the GNN-integrated architecture when the system scale increases. The slower learning speed can be attributed to the fact that MLP ignores the geographic relationship in the transportation network [11]. This makes the learning process less data-efficient and results in a lower sample efficiency.

C. Performance Under Different Scenarios

In this section, we follow the origin-based pricing mechanism and GNN-integrated network structure and further scrutinize the learned policy by conducting experiments with data from both San Francisco and three popular regions in NYC. The diverse demand and supply patterns observed in these four scenarios span a wide variety of real-world situations where the control framework can potentially be employed. These experiments are crucial for demonstrating the framework's adaptability across varied conditions and ensuring comparative analysis can yield valuable insights into the system control effectiveness.

TABLE I

CHARACTERISTICS OF THE FOUR SCENARIOS USED IN COMPARATIVE ANALYSIS. FOR THE BENCHMARKS GAP, THE FIRST COLUMN CORRESPONDS TO THE REWARD GAP AND THE SECOND COLUMN CORRESPONDS TO THE SERVED PASSENGER NUMBER GAP. A POSITIVE NUMBER INDICATES THAT THE EQUAL-DISTRIBUTION BENCHMARK HAS A HIGHER VALUE COMPARED TO THE NO-CONTROL BENCHMARK

Scenario	Benchmarks Gap (%)		Demand/Supply Ratio	Coefficient of Variation of Demand
	Reward	Served		
Man Middle	9.3	14.8	5.6	0.58
Man South	21.5	29.2	5.3	0.86
San Francisco	37.3	43.8	3.6	1.37
Brooklyn	73.8	78.5	1.95	1.63

TABLE I illustrates the four scenarios used in the comparative analysis. We characterize each scenario by three metrics: (1) the gap in gained rewards and served passengers between the no-control and equal-distribution benchmarks, (2) the ratio between the sum of potential demand and the number of vehicles, where the potential demand is the reference demand used in equation 10, and (3) the coefficient of variation of the demand. The coefficient of variation of the demand is calculated by dividing the standard deviation of the demand across all regions by the average regional demand. It reflects the variation of the demand across all regions. Among these three metrics, the benchmarks gap and coefficient of variation signify the distribution balance of demand across the network. Higher values for these two indicators suggest a more skewed demand distribution. The demand/supply ratio indicates the system supply level, with a higher ratio signifying a lower supply level.

TABLE II shows the performance of the learned policies of the four scenarios. Refer to Appendix C for the detailed training curves. For each scenario, we train three agents using three policies: (1) rebalancing, (2) dynamic pricing, and (3) the joint rebalancing and dynamic pricing approach. Observing the reward of the three policies across all four scenarios, it is noteworthy that, the joint approach consistently ensures the highest reward among the three. This result underscores the efficacy of combining rebalancing and dynamic pricing in achieving more reliable control under different scenarios.

TABLE II demonstrates the superiority of the joint rebalancing and pricing policy over the pricing-only policy across

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II

PERFORMANCE OF THE TRAINED POLICIES IN THE FOUR SCENARIOS PRESENTED IN TABLE I. WE PERFORM 10 TESTS FOR EACH POLICY AND REPORT THE AVERAGE PERFORMANCE WITH UPPER AND LOWER BOUNDS IN PARENTHESES. BOLD INDICATES THE BEST-PERFORMING POLICIES. IN THE COLUMNS, THE POLICY "REB" REPRESENTS REBALANCING AND "JOINT" REPRESENTS REBALANCING AND DYNAMIC PRICING POLICY

Scenario	NC	ED	PD	Heu	Reb	Pricing	Joint
Man Middle	37.4	40.9	40.3	39.5	40.2(±0.5)	49.7(±0.5)	**51.2(±0.5)**
Man South	33.8	41.1	40.8	39.3	41.0(±0.3)	50.4(±0.4)	**54.0(±0.4)**
San Francisco	13.3	18.3	17.5	16.9	18.3(±0.3)	17.8(±0.3)	**21.0(±0.5)**
Brooklyn	23.3	40.6	40.2	38.5	43.9(±0.7)	29.9(±1.0)	**53.8(±1.4)**

TABLE III

PERFORMANCE METRICS OF THE THREE POLICIES IN MAN SOUTH AND BROOKLYN. THE NUMBERS IN PARENTHESES INDICATE THE UPPER AND LOWER BOUNDS OF EACH METRIC FOR 10 TEST RUNS.IN THE ROWS, "PRICE" IS THE AVERAGE PRICE SCALAR SET BY THE AGENT, AND "WAIT" IS THE WAITING TIME OF THE SERVED PASSENGERS. ALL THE VALUES ARE AVERAGED ACROSS ALL THE REGIONS

	Man South			Brooklyn		
Policy	Reb	Pricing	Joint	Reb	Pricing	Joint
Reward	41.0(±0.3)	50.4(±0.4)	54.0(±0.4)	43.9(±0.7)	29.9(±1.0)	53.8(±1.4)
Rebalance Trips	915(±5)	——-	1446(±47)	2289(±35)	——-	1833(±40)
Price	——-	1.4(±0.1)	1.3(±0.0)	——-	1.4(±0.0)	1.4(±0.0)
Wait/min	1.7(±0.1)	0.6(±0.1)	0.4(±0.1)	0.9(±0.1)	0.6(±0.1)	0.2(±0.0)
Queue	19.6(±1.0)	4.7(±0.6)	3.8(±0.6)	3.2(±0.2)	1.5(±0.2)	0.5(±0.1)
Served Demand	7226(±33)	5666(±48)	6805(±72)	2707(±42)	1286(±30)	2364(±40)
Passenger Demand	8658(±64)	5768(±54)	6849(±67)	2988(±55)	1503(±35)	2411(±41)

different scenarios. In contrast, [44] finds that joint pricing and fleet rebalancing do not lead to significant profit improvements over pricing alone when demand can be accurately predicted. However, the benefits of a joint policy become much more evident when there are unexpected demand surges.

This discrepancy can be explained by the difference in the demand models. Our demand model (10) in Section IV-B allows the reference demand to scale up, allowing the rebalancing module to dynamically allocate vehicles to meet the additional demand triggered by the pricing module. This added flexibility makes the joint policy significantly more powerful than pricing alone. Conversely, in [44], the demand model only allows the reference model to be scaled down, which makes the actual demand always less than or equal to the reference demand. In this model, rebalancing primarily optimizes fleet usage for reduced demand rather than addressing additional demand. Under normal conditions, this limits the rebalancing module's contribution, as both policies serve demand capped by the reference. However, when there is an unexpected demand surge, the rebalancing module can effectively move vehicles to handle higher demand and achieve greater profitability and throughput.

In TABLE II, we also observe that the relative superiority between rebalancing and dynamic pricing changes in different scenarios. For Man Middle and Man South, the dynamic pricing policy outperforms the rebalancing policy, whereas this relationship is reversed for San Francisco and Brooklyn. To facilitate a more comprehensive and in-depth analysis of the underlying reasons, we focus on one scenario from each category, namely Man South and Brooklyn, and subject the learned policy to further testing and evaluation.

The results of the performance metrics are displayed in TABLE III. In Man South, the dynamic pricing policy obtains a higher reward, lower average waiting time, and queue length compared to the rebalancing policy. According to TABLE I, Man Middle and Man South exhibit the highest demand/supply ratio among the four scenarios. When demand significantly exceeds supply, at any time step, most vehicles are occupied, leaving only a small pool of available vehicles that can be influenced by rebalancing.

Moreover, TABLE III shows that, in Man South, the number of rebalancing trips is less than half that of Brooklyn, while the number of served passengers exceeds twice that of Brooklyn. This observation suggests that there is a lower need for frequent rebalancing efforts in Man South, as passenger demand greatly exceeds supply. In contrast, dynamic pricing has the flexibility to alter demand distribution by controlling trip prices without available vehicle constraints. In Man South, while an increase in price leads to a reduction in demand, the system grapples with insufficient supply. Consequently, the number of served demand in this case primarily depends on the supply level. Therefore, increasing prices helps elevate the average trip price without sacrificing a significant portion of served passengers, resulting in a higher reward.

In San Francisco and Brooklyn, the rebalancing policy outperforms the dynamic pricing policy. In these two scenarios, the demand/supply ratio is lower and the demand distribution is more unbalanced. This provides more room for the rebalancing policy to exert control. Additionally, a notable advantage of rebalancing over dynamic pricing is its ability to relocate the available vehicles from any region. In contrast, for dynamic pricing, if a large amount of vehicles is concentrated in a low-demand area, they tend to get stuck in this region as dynamic pricing alone may not induce sufficient demand to move all the stranded vehicles by matching them with passengers. If the potential demand in this region is already very low, it becomes very difficult to trigger demand high enough to relocate all the vehicles out of this region.

The aforementioned analysis of the limitation with dynamic pricing policy is substantiated by Fig. 4. Fig. 4 provides a visual depiction of the state evolution of the low-demand regions within Brooklyn under different policies. Under dynamic pricing policy, the number of vacant vehicles in these low-demand regions remains high (Fig. (a)), despite the agent's attempts to lower the prices to induce more demand (Fig. (b)). However, this issue is mitigated by the joint policy (Fig. (c)), which leverages the rebalancing module to relocate these vacant vehicles out from these low-demand regions (Fig. (d)). It is noteworthy that the price scalars in L1 and L4 are larger than 1, although they are expected to be lower to trigger more demand to move the sticking vehicles out. This happens because, in these low-demand regions, the demand is close to zero most of the time. As a result, changing pricing scalars in these regions does not have a significant impact on the reward. Thus, the agent does not focus on learning useful information from these regions.

To further inspect the behavior of the joint policy, we test the learned joint policy of Brooklyn. The system evolution by regions under the learned policy is shown in Fig. 5. For ease of interpretation, we focus our analysis on four regions out of the 14 regions in Brooklyn, comprising two high-demand regions

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

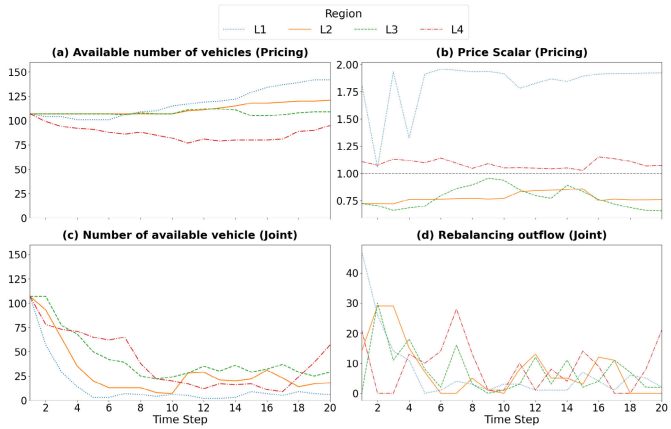

Fig. 4. Policy behavior and states for four low-demand regions in Brooklyn. (a) The number of available vehicles under the dynamic pricing policy, (b) the price scalar under the dynamic pricing policy, (c) the number of available vehicles under the joint policy, in which both rebalancing and dynamic pricing take effect, and (d) the rebalancing outflow under the joint policy. The joint policy solves the stuck vehicle problem in the pricing policy by leveraging rebalancing to relocate the available vehicles out from the low-demand regions.

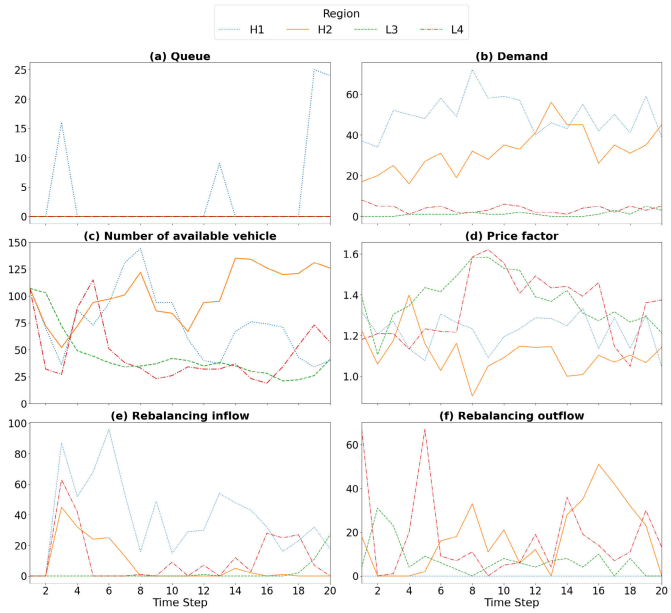

Fig. 5. Policy behavior and states for four regions in Brooklyn under the joint policy.

and two low-demand regions. For high-demand Region H1, the queue length increases at certain time steps (Fig. (a)), indicating a higher demand than supply in this region. To address this issue, the agent attempts to raise the price to reduce demand in this region, simultaneously rebalancing a high number of vehicles to this region (Fig. (d), (e)). In contrast, for high-demand Region H2, the available number of vehicles is high (Fig. (c)), particularly around time step 16. Consequently, the agent opts to set the price low in Region H2 while concurrently rebalancing vehicles away from this region (Fig. (d), (f)). For low-demand Regions L3 and L4, the agent focuses on reducing the stuck available vehicles in these two regions by relocating them elsewhere (Fig. (f)). These behaviors are coherent, underscoring the validity of the learned policy.

In summary, the performance of rebalancing and dynamic pricing policies heavily depends on the applied scenarios due to their distinct strengths and weaknesses. Specifically, rebalancing has control over all available vehicles in the system but faces limitations when demand overwhelms supply. Dynamic pricing, while not constrained by the number of available vehicles, is subject to potential demand constraints and cannot efficiently move vehicles that are stuck in low-demand areas. Combining rebalancing and dynamic pricing demonstrates the potential to address these limitations. As illustrated in TABLE II, the joint approach consistently exhibits superior performance across diverse demand scenarios.

D. Policy Generalization to Unseen Scenarios

In real-world systems, it is impractical and costly to fully train a controller online for each city. Therefore, the ability of the learned controller to generalize across different scenarios is a crucial aspect of the framework's real-world cost performance. We evaluate the generalization ability of the learned joint policies in the four scenarios introduced in Section V-C. We examine these learned policies on both the aforementioned four scenarios and other unseen scenarios, such as NYC Manhattan North (Man Middle), Washington DC, and Chicago. As discussed in Section III-C, the permutation invariance property of GNNs ensures that the learned policy is unaffected by the size or ordering of the graph. Instead, the GNN focuses on capturing the relationships between neighboring nodes. This property allows us to directly employ the learned policies from each one of the four scenarios to other unseen scenarios without additional training.

TABLE IV displays the generalization performance of the learned policies discussed in Section V-C. The values in red indicate generalization performance worse than the no-control benchmark, implying that the learned policy fails to generalize any learned information to the tested scenario. In most of the tested scenarios, the learned policy from the four scenarios at least outperforms the no-control and equal-distribution benchmark, with some achieving performance on par with the fully online trained policies. These findings demonstrate that the learned correlation between the system state and optimal action from one scenario is generalizable and transferable to other unseen scenarios.

The instances of low generalization performance result from mismatches between the training data distribution and test data distribution. For instance, the learned policy in San Francisco exhibits low generalization ability to Man South and Chicago due to significant disparities in demand patterns. San Francisco has the lowest demand in all the scenarios, whereas Man South and Chicago are among the top, which is around 5 times higher than San Francisco. As a result, the system state of Man South and Chicago falls outside the distribution on which the San Francisco policy is trained, leading to poor policy performance. It is also noteworthy that for the instances where Brooklyn is the tested scenario, only the learned policy in San Francisco outperforms the benchmarks. Further examination of the demand patterns reveals that Brooklyn and San Francisco exhibit polarized demand, with some regions experiencing high demand and others low demand. In contrast, in Man South and Man Middle, the demand is more evenly distributed

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE IV

GENERALIZATION PERFORMANCE OF THE LEARNED POLICY FROM THE FOUR DISCUSSED SCENARIOS IN NYC AND SAN FRANCISCO. THE "NO CONTROL" AND "EQUAL DISTRIBUTION" COLUMNS INDICATE THE BENCHMARK REWARDS OF EACH SCENARIO, THE "FULLY TRAINED" COLUMN PRESENTS THE PERFORMANCE OF THE CORRESPONDING FULLY ONLINE TRAINED POLICY, THE "SOURCE POLICY SCENARIO" COLUMN IS THE SCENARIO ON WHICH WE TRAIN THE DEPLOYED POLICY, AND THE "TEST REWARD" POLICY IS THE PERFORMANCE OF THIS DEPLOYED POLICY ON THE CORRESPONDING UNSEEN SCENARIO. IN THE "TEST REWARD" COLUMN, THE NUMBERS IN **BOLD** INDICATE THE BEST GENERALIZATION PERFORMANCE IN EACH SCENARIO, AND THE NUMBERS IN RED REPRESENT THE CASES WHERE THE POLICIES FAIL TO GENERALIZE

Target Scenario	NC	ED	Fully Trained	Source Policy Scenario	Test Reward
Man Middle	37.4	40.9	51.2(±0.5)	Man South	48.7(±0.6)
				San Francisco	44.1(±1.5)
				Brooklyn	**50.0(±0.5)***
Man South	33.8	41.1	54.0(±0.4)	San Francisco	24.9(±2.1)
				Brooklyn	**53.9(±0.4)†**
				Man Middle	52.6(±0.4)
San Francisco	13.3	18.3	21.0 (±0.5)	Man Middle	**20.9(±0.4)***
				Man South	20.8(±0.4)
				Brooklyn	20.4(±0.4)
Brooklyn	23.3	40.6	53.8(±1.4)	Man South	18.4(±1.1)
				San Francisco	**41.1(±1.3)***
				Man Middle	19.6(±1.4)
Man North	28.0	28.8	39.0 (±0.3)	Man South	35.9(±0.3)
				San Francisco	**37.7(±0.4)**
				Brooklyn	36.6(±0.5)
				Man Middle	37.6(±0.2)
Washington	14.3	16.7	31.6 (±0.1)	Man South	**26.7(±0.3)**
				San Francisco	20.2(±0.9)
				Brooklyn	26.5(±0.4)
				Man Middle	25.1(±0.2)
Chicago	35.1	41.0	63.4 (±1.4)	Man South	**62.5(±1.5)**
				San Francisco	4.9(±1.1)
				Brooklyn	61.5(±1.1)
				Man Middle	60.2(±1.3)

along the spectrum. This can also be seen from TABLE I where San Francisco and Brooklyn display lower benchmarks gap and demand variation coefficients compared to the other two scenarios.

The outcomes of the generalization experiments demonstrate the transferability of the learned policy to scenarios with similar state patterns as the training scenario. Furthermore, the analysis of these results shows that the three metrics proposed to characterize the different scenarios in TABLE I serve as a reliable proxy for measuring the similarity between scenarios. These metrics also provide a useful estimate of how well the policy will transfer across two scenarios. Additionally, they underscore the significance of addressing the out-of-distribution problem to improve the generalization ability of the learned policy to more diverse scenarios. However, tackling this problem is beyond the scope of this paper. Interested readers can refer to [60] for a comprehensive introduction to this topic.

Policy transferability is not only crucial for avoiding the impracticality of online training for each scenario but also provides a feasible solution for cities lacking an available online training environment. Another promising approach to address the issue of unavailable online training resources is to train the agent using offline static datasets. In the next section, we examine the performance of offline RL on the joint rebalancing and dynamic pricing problem and compare it to the generalization performance of the learned policies discussed in this section.

E. Learning From Offline Data

In this section, we evaluate the framework through the proposed offline RL approach using data from San Francisco, Man Middle, Man South, and Brooklyn. The training data for offline RL is the state and action pairs collected from the behavior policy. To prepare the offline training dataset for each scenario, we first trained the agent to achieve around 75% and 90% of the reward of the fully trained state. The learned policy is suboptimal, and we used it as the behavior policy to collect data through online interaction with the environment. Besides the partially trained stochastic policy, we also collect datasets by the deterministic heuristic policy mentioned in Section V-C. As the desired vehicle distributions and price multipliers are not available in the trip dataset, we use the heuristic as a proxy for the policy that is deployed behind the trip dataset. A detailed explanation of how the heuristic generates the price multipliers can be found in Appendix D. The three datasets are used to perform offline learning following the CQL and IQL algorithms discussed in Section III-B.

TABLE V shows that through learning from offline data collected by both the suboptimal and heuristic policy, the agents achieve superior performance compared to their corresponding behavior policy, and CQL exhibits an overall better performance in comparison to IQL. Specifically, in comparison to the experts that have full information about the system and are fully trained online, the reward from the CQL-trained policy trained on 75% reward dataset is only 7.1% (Man Middle), 9.1% (Man South), 5.1% (San Francisco), and 11% lower, with an even narrower gap in the results of the policy trained by the 90% dataset. These results demonstrate that the offline RL method is an effective approach to leverage static datasets to extract useful information. By "stitching" the information from the static datasets, the trained policy holds the potential to outperform the policy represented by the datasets.

The results in TABLE V also underscore the influence of dataset characteristics on the outcomes of offline RL. Notably, a behavior policy yielding higher rewards does not invariably translate to a better performance of the offline-trained agent. As indicated in the TABLE, in Man Middle, although the heuristic policy exhibits a much lower reward compared to the 75% and 90% trained policies, the offline-learned policy derived from the dataset collected by the heuristic policy outperforms 75% and is very close to 95%. We hypothesize that this discrepancy arises because the low-reward samples are also important in training the RL agent, as they help the agent grasp the characteristics of low-quality actions and understand the nuances of suboptimal decisions.

If we compare the performance of the offline-learned policies in TABLE V to the generalization performance of the learned policy on Man Middle, Man South, San Francisco, and Brooklyn in TABLE IV, we can observe that for Man Middle, San Francisco, and Brooklyn, the best-performing offline-learned policies (numbers with * in TABLE V) achieve comparable or outperform the generalization performance of the polices learned in Brooklyn, Man Middle, and San Francisco (numbers with * in TABLE IV). However, for Man South, the generalization performance of the policy learned in Brooklyn (number with †in TABLE IV) outperforms

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE V

REWARD OF THE BEHAVIOR POLICY, CQL-TRAINED POLICY, IQL-TRAINED POLICY, AND EXPERT POLICY (THOUSANDS OF DOLLARS). THE EXPERT IS THE POLICY THAT IS FULLY TRAINED ONLINE. THE NUMBERS IN BOLD INDICATE BETTER OFFLINE-TRAINED POLICY BETWEEN CQL AND IQL

Dataset	Scenario	Behavior Policy	CQL	IQL	Expert
75%	Man Middle	43.6(±0.4)	**47.5(±0.4)**	45.2(±0.5)	51.2(±0.5)
	Man South	40.0(±0.8)	**50.4(±0.4)**	49.2(±0.5)	54.0(±0.4)
	San Francisco	16.9(±0.2)	**20.3(±0.3)**	19.4(±0.3)	21.0(±0.5)
	Brooklyn	39.0(±2.1)	**47.9(±1.0)**	41.5(±2.0)	53.8(±1.4)
90%	Man Middle	46.1(±0.5)	**49.8(±0.3)***	48.5(±0.5)	51.2(±0.5)
	Man South	48.5(±0.6)	**51.4(±0.5)**†	50.5(±0.4)	54.0(±0.4)
	San Francisco	19.0(±0.4)	**20.6(±0.5)***	20.2(±0.3)	21.0(±0.5)
	Brooklyn	47.2(±2.1)	**52.2(±1.1)***	47.8(±1.0)	53.8(±1.4)
Heuristic	Man Middle	39.5(±0.8)	**49.6(±0.4)**	39.8(±0.4)	51.2(±0.5)
	Man South	39.3(±0.8)	**46.5(±0.9)**	37.8(±0.9)	54.0(±0.4)
	San Francisco	16.9(±0.5)	**20.0(±0.4)**	17.9(±0.3)	21.0(±0.5)
	Brooklyn	38.5(±1.3)	**40.4(±0.8)**	34.0(±1.5)	53.8(±1.4)

the best-performing offline-learned policy (number with †in TABLE V). These results reveal that both generalization and offline RL can serve as viable solutions whenever online training is not available. In other words, when online training is not feasible for a city, one could consider leveraging the learned policy from other cities or conducting offline training if the static dataset is available. In addition, these two methods can also be used as a pre-training to speed up and facilitate further online fine-tuning. In our experiments, the generalization result shows an overall slightly superior performance than offline RL, but in reality, the choice between the two approaches should depend on the availability of static datasets.

It is also important to highlight that a key advantage of the offline RL method is the significant improvement of the sample efficiency. For example, in our experiments, while training 1000 epochs online requires approximately 20000 interactions with the environment, offline RL is solely trained on a pre-collected fixed-size dataset, resulting in a significant improvement in the training efficiency. In addition, the non-interactive learning process makes the offline training parallelizable, enabling more efficient use of the computational resources and enhancing the training efficiency.

VI. CONCLUSION

Among the extensive research on pricing and rebalancing problems in MoD systems across both operations research and RL fields, the nature of the OD-based decision space consistently presents scalability challenges. This bottleneck hinders the application of the proposed models to real-world scale problems. In this paper, we develop a bi-level RL-based framework where control is exerted through dynamic trip pricing and vehicle rebalancing, with actions designed on a node-based level. This framework is applicable under both online and offline problem settings. As can be seen from the experiments on the comparison between origin and OD-based price, we demonstrate that origin-based pricing can deliver comparable performance to OD-based pricing while facilitating higher sample efficiency and faster convergence. Furthermore, we enhance data efficiency and convergence rates by proposing the utilization of GNNs as policy and action value approximators compared to the extensively adopted MLP-based policy in the same line of research.

Experimental results across various scenarios in NYC and San Francisco highlight the robustness of the joint policy in enabling more effective system control in comparison to dynamic pricing or rebalancing policies alone. The dynamic pricing or rebalancing policies exhibit different limitations across different scenarios, depending on the characteristics of the latter. In contrast, the joint policy shows remarkable adaptability by switching between the two control mechanisms - rebalancing or dynamic pricing - when one faces limited control ability. It is also demonstrated to be effective in leveraging the advantages of the two mechanisms and making them in tandem to further maximize profit.

The learned joint policies from NYC and San Francisco are deployed to unseen scenarios to assess their generalization ability. The findings indicate that the learned policy under our proposed framework can achieve good generalization performance, but still encounters challenges with the out-of-distribution scenarios that significantly deviate from the trained ones. Moreover, the analysis of the learned policies from different scenarios reveals that the three axes we propose to characterize different scenarios - benchmarks gap, demand/supply ratio, and demand variation coefficient - provide a good intuition about the generalization performance across them.

Lastly, we show that the developed framework can also be trained by offline RL. The results indicate that the offline-trained agents outperform the corresponding behavior policies, and sometimes even achieve performance on par with the online-trained agents. The agents also learn from the datasets that are not collected by a central policy, as in the case of the heuristic datasets. The results of offline RL prove the possibility of applying our proposed framework in real-world applications, where a substantial amount of static data is available and online learning through interactions with a real transportation system can be costly.

We also notice several open questions worthy of further exploration and could serve as potential directions for future research. Firstly, our proposed training framework does not always guarantee generalization to different scenarios. To enhance generalization ability, future research could explore the concept of "Learning to Learn" [61], also known as meta-learning in the machine learning community. This approach treats different problems as training instances and aims to enable the optimizer to learn the underlying problem structures of various scenarios [62]. Another possibility is to experiment among different permutation-invariant structures, for example, different types of GNN [63], [64], or transformer-based architecture [65] to find the structure exhibiting the best generalization ability. Secondly, in our offline RL experiments, we periodically test the learning agent online and save the

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

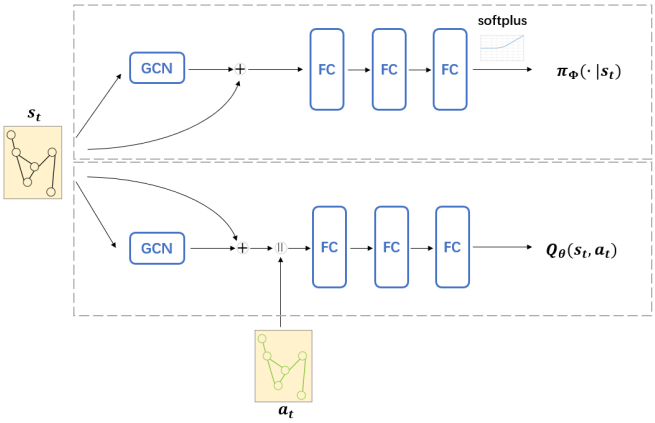

Fig. 6. GNN-integrated neural network architecture for actor and critic.

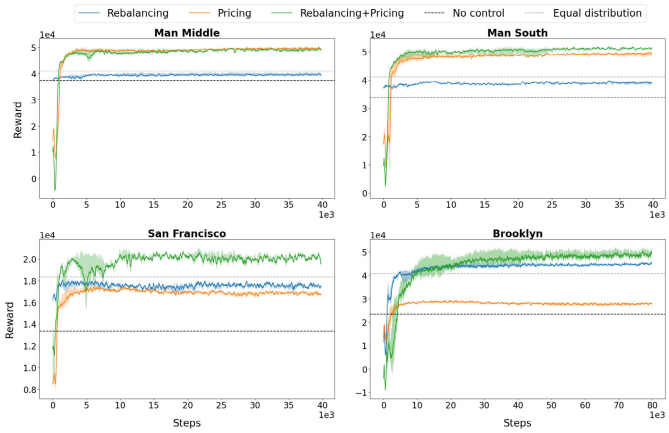

Fig. 7. Training curve of the four scenarios. Each sub figure includes the training reward of three policies: rebalancing, pricing, and joint, alongside two benchmarks: no-control and equal-distribution.

best-learned policy according to the test reward. However, in real-world offline scenarios, there may not be an environment available to test the agent's performance. Thus, designing offline learning metrics that facilitate determining the optimal learning state is essential to enable interaction-free offline RL. This direction of research holds promise for advancing the applicability of offline RL methods in real-world settings. Thirdly, the findings from the policy generalization and offline RL experiments underscore the transferability of the learned policies and the effectiveness of offline RL in tackling the joint learning problem. This suggests that meta-offline RL could offer a promising avenue to combine the strengths of both approaches, leading to improved learning outcomes when offline datasets from diverse scenarios are available. The combined approach also holds the potential to facilitate more efficient online fine-tuning. Lastly, the RL-based approach can be explored to incorporate other centralized decision-making aspects, such as dispatching and charging of electric vehicles. Given its capacity to coordinate different operations for more effective control of the complex system, this avenue holds promise for enabling comprehensive control through the RL method.

Appendix A
Network Architecture

Fig 6 shows the architecture of the GNN-integrated neural network. The upper frame represents the actor and the lower frame represents the critic.

Appendix B
Rebalancing Model

The rebalancing model is the same as the one used in [11]. This model seeks to derive the rebalancing flows $\{y_{ij}^t\}_{(i,j)\in\mathcal{E}}$ with a minimized rebalancing-cost:

$$min \sum_{(i,j)\in\mathcal{E}} c_{ij}^t y_{ij}^t \tag{13a}$$

$$s.t. \quad \sum_{j\neq i}(y_{ji}^t - y_{ij}^t) + m_i^t \geq \widehat{m}_i^t, i \in \mathcal{V} \tag{13b}$$

$$\sum_{j\neq i} y_{ij}^t \leq m_i^t, i \in \mathcal{V} \tag{13c}$$

where $\widehat{m}_i^t$ denotes the number of desired vehicles. Objective (13a) minimizes the rebalancing cost. Constraint (13b) ensures the desired vehicle number is satisfied, and constraint (13c) limits the rebalancing flow by the number of available vehicles. $\widehat{m}_i^t$ in constraint (13b) is calculated by $\widehat{m}_i^t = \lfloor w_i^t \sum_{i\in\mathcal{V}} m_i^t \rfloor$.

Appendix C
Training Curve of the Four Scenarios

Fig 7 shows the reward curves during the training process of the four scenarios. The x-axis represents the training step. In our RL setup, one episode comprises 20 training steps.

Appendix D
Heuristic Policy for Offline Training

In the heuristic policy for the joint rebalancing and pricing problem, the prices are decided by a heuristic first, then the desired vehicle distributions are derived from the rebalancing flows, which are calculated by another heuristic, based on the results of the pricing actions. The pricing heuristic follows similar ideas as the SDE algorithm presented in [56]. In SDE, the price is adjusted only when the demand surpasses the supply, and the price can be set at most three times as high as the base price. To make it more consistent with our problem setting, we modify the heuristic to update the price both when the passenger is lower or higher than the supply. Moreover, the price can only be set twice as high as the base price. The pricing heuristic results in the following format:

$$p_{ij} = \begin{cases} \overline{p}_{ij} \cdot (1 + e^{|m_i - d_i|}), & \text{if } m_i \geq d_i \\ \overline{p}_{ij} \cdot (1 - e^{|d_i - m_i|}), & \text{otherwise} \end{cases}, \tag{14a}$$

where p_{ij} is the adjusted price from i to j, $\overline{p}_{ij}$ is the base price, m_i is the available number of vehicles in region i, and d_i is the demand from the region i. Intuitively, the price will be reduced if there is an oversupply and be increased with the overdemand.

After the price is decided by the aforementioned pricing heuristic, the passengers will be generated and matched with the available vehicles. Subsequently, the available number of vehicles and queue length will be updated. The rebalancing flows are then calculated based on the newly available

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

vehicles and queue length information by the individual assignment-based rebalancing model described in [57] as follows:

$$min \sum_{v \in \mathcal{V}_{idle}} \sum_{r \in \mathcal{Q}} \tau_{v,r} y_{v,r} \tag{15a}$$

$$s.t. \quad \sum_{v \in \mathcal{V}_{idle}} \sum_{r \in \mathcal{Q}} y_{v,r} = min(|\mathcal{V}_{idle}, |\mathcal{Q}|), \tag{15b}$$

$$0 \leq y_{v,r} \leq 1, \tag{15c}$$

where $\mathcal{V}_{idle}$ is the idle vehicle set, $\mathcal{Q}$ is the passenger queue set, $|\cdot|$ denotes the number of elements in set, and $y_{v,r}$ is the decision variable indicating the individual assignments. Assign vehicle v to passenger r translates into rebalancing vehicle v to the origin of passenger r. The rebalancing flows are subsequently converted to the desired vehicle distributions through an inversion process as in [66].

References

[1] H. Ritchie and M. Roser, "Urbanization," *Our World Data*, 2018. [Online]. Available: https://ourworldindata.org/urbanization

[2] D. Schrank, T. Lomax, and S. Turner, "Urban Mobility Report," Texas Transp. Inst., Texas A&M Univ., Tech. Rep. UMR-2011, College Station, TX, USA, pp. 43–50, Sep. 2011. [Online]. Available: https://static.tti.tamu.edu/tti.tamu.edu/documents/umr/archive/mobility-report-2011-wappx.pdf

[3] J. B. Oke et al., "Evaluating the systemic effects of automated mobility-on-demand services via large-scale agent-based simulation of auto-dependent prototype cities," *Transp. Res. A, Policy Pract.*, vol. 140, pp. 98–126, Oct. 2020.

[4] S. Oh, R. Seshadri, C. L. Azevedo, N. Kumar, K. Basak, and M. Ben-Akiva, "Assessing the impacts of automated mobility-on-demand through agent-based simulation: A study of Singapore," *Transp. Res. A, Policy Pract.*, vol. 138, pp. 367–388, Aug. 2020.

[5] S. Oh, A. F. Lentzakis, R. Seshadri, and M. Ben-Akiva, "Impacts of automated mobility-on-demand on traffic dynamics, energy and emissions: A case study of Singapore," *Simul. Model. Pract. Theory*, vol. 110, Jul. 2021, Art. no. 102327.

[6] B. Boyacı, K. G. Zografos, and N. Geroliminis, "An integrated optimization-simulation framework for vehicle and personnel relocations of electric carsharing systems with reservations," *Transp. Res. B, Methodol.*, vol. 95, pp. 214–237, Jan. 2017.

[7] X. Guo, Q. Wang, and J. Zhao, "Data-driven vehicle rebalancing with predictive prescriptions in the ride-hailing system," *IEEE Open J. Intell. Transp. Syst.*, vol. 3, pp. 251–266, 2022.

[8] D. Jorge, G. Molnar, and G. H. de Almeida Correia, "Trip pricing of one-way station-based carsharing networks with zone and time of day price variations," *Transp. Res. B, Methodol.*, vol. 81, pp. 461–482, Nov. 2015.

[9] L. Wang and W. Ma, "Pricing approach to balance demands for one-way car-sharing systems," in *Proc. 22nd IEEE Intell. Transp. Syst. Conf. (ITSC)*, Jun. 2019, pp. 1697–1702.

[10] R. Karamanis, P. Angeloudis, A. Sivakumar, and M. Stettler, "Dynamic pricing in one-sided autonomous ride-sourcing markets," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2018, pp. 3645–3650.

[11] D. Gammelli, K. Yang, J. Harrison, F. Rodrigues, F. C. Pereira, and M. Pavone, "Graph neural network reinforcement learning for autonomous mobility-on-demand systems," in *Proc. 60th IEEE Conf. Decis. Control (CDC)*, Dec. 2021, pp. 2996–3003.

[12] J. Holler et al., "Deep reinforcement learning for multi-driver vehicle dispatching and repositioning problem," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2019, pp. 1090–1095.

[13] C. Fluri, C. Ruch, J. Zilly, J. Hakenberg, and E. Frazzoli, "Learning to operate a fleet of cars," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Oct. 2019, pp. 2292–2298.

[14] J. Wen, J. Zhao, and P. Jaillet, "Rebalancing shared mobility-on-demand systems: A reinforcement learning approach," in *Proc. IEEE 20th Int. Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2017, pp. 220–225.

[15] X. Tang, M. Li, X. Lin, and F. He, "Online operations of automated electric taxi fleets: An advisor-student reinforcement learning framework," *Transp. Res. C, Emerg. Technol.*, vol. 121, Dec. 2020, Art. no. 102844.

[16] J. Song, Y. J. Cho, M. H. Kang, and K. Y. Hwang, "An application of reinforced learning-based dynamic pricing for improvement of ridesharing platform service in Seoul," *Electronics*, vol. 9, no. 11, p. 1818, Nov. 2020.

[17] C. Chen, F. Yao, D. Mo, J. Zhu, and X. Chen, "Spatial–temporal pricing for ride-sourcing platform with reinforcement learning," *Transp. Res. C, Emerg. Technol.*, vol. 130, Sep. 2021, Art. no. 103272.

[18] J. Huang et al., "Deep reinforcement learning-based trajectory pricing on ride-hailing platforms," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 3, pp. 1–19, Jun. 2022.

[19] T. Enders, J. Harrison, M. Pavone, and M. Schiffer, "Hybrid multi-agent deep reinforcement learning for autonomous mobility on demand systems," in *Proc. Learn. Dyn. Control Conf.*, Jan. 2022, pp. 1284–1296.

[20] Y. Kim, V. Jayawardana, and S. Samaranayake, "Learning-augmented vehicle dispatching with slack times for high-capacity ride-pooling," SSRN Preprint, 4801437, Jan. 2024.

[21] A. O. Al-Abbasi, A. Ghosh, and V. Aggarwal, "DeepPool: Distributed model-free algorithm for ride-sharing using deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 12, pp. 4714–4727, Dec. 2019.

[22] M. Guériau and I. Dusparic, "SAMoD: Shared autonomous mobility-on-demand using decentralized reinforcement learning," in *Proc. IEEE Intell. Transp. Syst. Conf. (ITSC)*, Nov. 2018, pp. 1558–1563.

[23] M. Pavone, S. L. Smith, E. Frazzoli, and D. Rus, "Robotic load balancing for mobility-on-demand systems," *Int. J. Robot. Res.*, vol. 31, no. 7, pp. 839–854, 2012.

[24] J. Warrington and D. Ruchti, "Two-stage stochastic approximation for dynamic rebalancing of shared mobility systems," *Transp. Res. C, Emerg. Technol.*, vol. 104, pp. 110–134, Jul. 2019.

[25] M. Tsao, R. Iglesias, and M. Pavone, "Stochastic model predictive control for autonomous mobility on demand," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Nov. 2018, pp. 3941–3948.

[26] R. Zhang and M. Pavone, "Control of robotic mobility-on-demand systems: A queueing-theoretical perspective," *Int. J. Robot. Res.*, vol. 35, nos. 1–3, pp. 186–203, Jan. 2016.

[27] R. Iglesias, F. Rossi, K. Wang, D. Hallac, J. Leskovec, and M. Pavone, "Data-driven model predictive control of autonomous mobility-on-demand systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 6019–6025.

[28] X. Guo, N. S. Caros, and J. Zhao, "Robust matching-integrated vehicle rebalancing in ride-hailing system with uncertain demand," *Transp. Res. B Methodol.*, vol. 150, pp. 161–189, Aug. 2021.

[29] C. Schmidt, D. Gammelli, F. C. Pereira, and F. Rodrigues, "Learning to control autonomous fleets from observation via offline reinforcement learning," in *Proc. Eur. Control Conf. (ECC)*, Jun. 2024, pp. 1399–1406.

[30] S. Banerjee, R. Johari, and C. Riquelme, "Pricing in ride-sharing platforms: A queueing-theoretic approach," in *Proc. 16th ACM Conf. Econ. Comput. (EC)*. New York, NY, USA: Association for Computing Machinery, 2015, p. 639. [Online]. Available: https://doi.org/10.1145/2764468.2764527

[31] K. Bimpikis, O. Candogan, and D. Saban, "Spatial pricing in ride-sharing networks," *Oper. Res.*, vol. 67, no. 3, pp. 744–769, May 2019.

[32] F. Afifah and Z. Guo, "Spatial pricing of ride-sourcing services in a congested transportation network," *Transp. Res. C, Emerg. Technol.*, vol. 142, Sep. 2022, Art. no. 103777.

[33] C. Lei, Z. Jiang, and Y. Ouyang, "Path-based dynamic pricing for vehicle allocation in ridesharing systems with fully compliant drivers," *Transp. Res. B, Methodol.*, vol. 132, pp. 60–75, Jan. 2020.

[34] M. Xu, Q. Meng, and Z. Liu, "Electric vehicle fleet size and trip pricing for one-way carsharing services considering vehicle relocation and personnel assignment," *Transp. Res. B, Methodol.*, vol. 111, pp. 60–82, May 2018.

[35] R. Xie, W. Wei, Q. Wu, T. Ding, and S. Mei, "Optimal service pricing and charging scheduling of an electric vehicle sharing system," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 78–89, Jan. 2020.

[36] T. Wu, A. D. Joseph, and S. J. Russell, "Automated pricing agents in the on-demand economy," Dept. Electrical Engineering and Computer Sciences, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2016-57, 2016.

[37] B. Turan, R. Pedarsani, and M. Alizadeh, "Dynamic pricing and fleet management for electric autonomous mobility on demand systems," *Transp. Res. Part C: Emerg. Technol.*, vol. 121, Dec. 2020, Art. no. 102829.

[38] A. Wang and B. Turan, "Multi-agent reinforcement learning for dynamic pricing and fleet management in autonomous mobility-on-demand systems," in *Proc. Int. Telemetry Conf.* Palmdale, CA, USA: International Foundation for Telemetering, 2022.

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[39] Z. Lei and S. V. Ukkusuri, "Scalable reinforcement learning approaches for dynamic pricing in ride-hailing systems," *Transp. Res. Part B: Methodol.*, vol. 178, Dec. 2023, Art. no. 102848.

[40] R. Lu, G. H. D. A. Correia, X. Zhao, X. Liang, and Y. Lv, "Performance of one-way carsharing systems under combined strategy of pricing and relocations," *Transportmetrica B: Transp. Dyn.*, vol. 9, no. 1, pp. 134–152, Jan. 2021.

[41] G. Pantuso, "Exact solutions to a carsharing pricing and relocation problem under uncertainty," *Comput. Operations Res.*, vol. 144, Aug. 2022, Art. no. 105802.

[42] S. Wollenstein-Betech, I. Ch. Paschalidis, and C. G. Cassandras, "Joint pricing and rebalancing of autonomous mobility-on-demand systems," in *Proc. 59th IEEE Conf. Decis. Control (CDC)*, Dec. 2020, pp. 2573–2578.

[43] R. Iacobucci and J.-D. Schmöcker, "Dynamic pricing for ride-hailing services considering relocation and mode choice," in *Proc. 7th Int. Conf. Models Technol. Intell. Transp. Syst. (MT-ITS)*, Jun. 2021, pp. 1–6.

[44] Z. Lai and S. Li, "Spatiotemporal pricing and fleet management of autonomous mobility-on-demand networks: A decomposition and dynamic programming approach with bounded optimality gap," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7057–7069, Jul. 2024.

[45] C. Lei, Z. Jiang, and Y. Ouyang, "Path-based dynamic pricing for vehicle allocation in ridesharing systems with fully compliant drivers," *Transp. Res. Proc.*, vol. 38, pp. 77–97, Jan. 2019.

[46] L. Al-Kanj, J. Nascimento, and W. B. Powell, "Approximate dynamic programming for planning a ride-hailing system using autonomous fleets of electric vehicles," *Eur. J. Oper. Res.*, vol. 284, no. 3, pp. 1088–1106, Aug. 2020.

[47] D. Gammelli, I. Peled, F. Rodrigues, D. Pacino, H. A. Kurtaran, and F. C. Pereira, "Estimating latent demand of shared mobility through censored Gaussian processes," *Transp. Res. C, Emerg. Technol.*, vol. 120, Nov. 2020, Art. no. 102775.

[48] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1179–1191.

[49] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.

[50] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," 2020, *arXiv:2005.01643*.

[51] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," 2021, *arXiv:2110.06169*.

[52] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[53] D. Gammelli, J. Harrison, K. Yang, M. Pavone, F. Rodrigues, and F. C. Pereira, "Graph reinforcement learning for network control via bi-level optimization," 2023, *arXiv:2305.09129*.

[54] J. M. Rose and D. A. Hensher, "Demand for taxi services: New elasticity evidence," *Transportation*, vol. 41, no. 4, pp. 717–743, Jul. 2014.

[55] D. Gammelli, K. Yang, J. Harrison, F. Rodrigues, F. Pereira, and M. Pavone, "Graph meta-reinforcement learning for transferable autonomous mobility-on-demand," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2913–2923.

[56] Y. Tong, L. Wang, Z. Zhou, L. Chen, B. Du, and J. Ye, "Dynamic pricing in spatial crowdsourcing: A matching-based approach," in *Proc. Int. Conf. Manage. Data*, May 2018, pp. 773–788.

[57] J. Alonso-Mora, S. Samaranayake, A. Wallar, E. Frazzoli, and D. Rus, "On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 3, pp. 462–467, Jan. 2017.

[58] A. Shehzad et al., "Graph transformers: A survey," 2024, *arXiv:2407.09777*.

[59] C. Chen et al., "A survey on graph neural networks and graph transformers in computer vision: A task-oriented perspective," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10297–10318, Dec. 2024.

[60] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, "Meta-learning in neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 5149–5169, Sep. 2021.

[61] S. Thrun and L. Pratt, "Learning to learn: Introduction and overview," in *Learning to Learn*. Cham, Switzerland: Springer, 1998, pp. 3–17.

[62] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. de Freitas, "Learning to learn by gradient descent by gradient descent," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 29. MIT Press, 2016, pp. 3981–3989.

[63] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv:1806.01261*.

[64] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," 2017, *arXiv:1709.05584*.

[65] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[66] C. Schmidt, D. Gammelli, J. Harrison, M. Pavone, and F. Rodrigues, "Offline hierarchical reinforcement learning via inverse optimization," 2024, *arXiv:2410.07933*.

Xinling Li received the bachelor's degree in traffic engineering from Tongji University in 2021 and the master's degree in transportation and mobility from EPFL in 2024. She is currently pursuing the degree in transportation with MIT, supervised by Prof. Jinhua Zhao and Prof. Gioele Zardini. During her master's studies, she had the opportunity to work at the Machine Learning for Smart Mobility (MLSM) Laboratory, DTU, where she conducted her master's project on learning the joint rebalancing and dynamic pricing policies in autonomous mobility-on-demand system under the supervision of Prof. Filipe Rodrigues. Her research interests are on the control and optimization of smart mobility systems.

Carolin Schmidt received the master's degree (Hons.) in electrical power engineering and operations research from RWTH Aachen University. She is a Post-Doctoral Scholar at the Machine Learning for Smart Mobility Laboratory, Technical University of Denmark (DTU), where her research includes learning-based control for autonomous mobility systems and climate change adaptation planning. As part of her Ph.D., she was involved in an EU-wide project on the real-world deployment of autonomous shuttles. During her time as a Visiting Researcher at the Autonomous Systems Laboratory, Stanford University, she worked on integrating deep reinforcement learning (RL) with established optimization and control methods to create efficient, robust, and safe RL, contributing to applications in large-scale transportation systems, operations research, and robotics.

Daniele Gammelli received the Ph.D. degree in machine learning and mathematical optimization from the Department of Technology, Management and Economics, Technical University of Denmark. He is a Post-Doctoral Scholar with the Autonomous Systems Laboratory, Stanford University, and a Research Fellow at the Center for Aerospace Autonomy Research (CAESAR) at Stanford. His research focuses on developing learning-based solutions that enable the deployment of future autonomous systems in complex environments, with an emphasis on large-scale robotic networks, aerospace systems, and future mobility systems. His research interests include deep reinforcement learning, imitation learning, foundation models, graph representation learning, and control techniques leveraging these tools.

Filipe Rodrigues is an Associate Professor at the Machine Learning for Smart Mobility (MLSM) Laboratory, Technical University of Denmark (DTU), where his research focuses primarily on the use of machine learning to understand and optimize urban mobility and human behavior. Previously, he was an H. C. Ørsted Marie-Skodowska Curie Actions (COFUND) Post-Doctoral Fellow, working on spatio-temporal models of mobility demand with emphasis on modelling uncertainty and the effect of special events. He has published more than 60 research articles in leading conferences and journals in both transportation and machine learning. His research interests span machine learning, reinforcement learning, intelligent transportation systems, and urban mobility.

Authorized licensed use limited to: Technical University of Denmark DTU Library. Downloaded on July 07,2025 at 06:13:49 UTC from IEEE Xplore. Restrictions apply.